



CogniWell

Intelligent Cognitive Alarm & Behavioral Health Platform

An AI-powered platform that helps users build consistent wake-up habits, overcome sleep inertia, and maximize daily productivity through personalized cognitive challenges required before alarm dismissal.

Project Title	CogniWell — Intelligent Cognitive Alarm Platform
Prepared By	Samhitha Sura
Document Type	Project Report (Technical & Functional Overview)

1. Project Overview

Traditional alarms fail because they allow low-effort snoozing and instant dismissal while the brain is still experiencing **sleep inertia** — the grogginess and impaired cognitive function that follows waking. **CogniWell** solves this problem by requiring users to complete interactive, difficulty-calibrated cognitive tasks (math problems, memory cards, logic riddles) before an alarm can be dismissed.

The platform tracks behavioral wake-up metrics, calculates a daily **Weighted Scoring Model** in PostgreSQL, generates dynamic 7-day weekly recommendations, and enables seamless communication between users and assigned Wellness Coaches — turning a simple alarm into a behavioral health and productivity tool.

Target Users

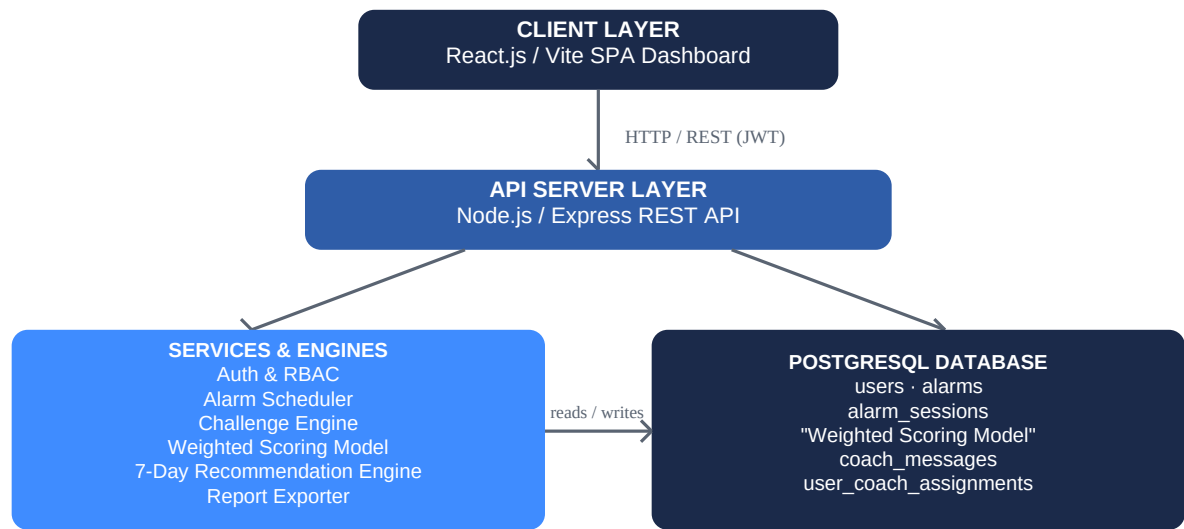
Students, working professionals, fitness enthusiasts, productivity-focused individuals, and wellness platforms seeking a structured wake-up and habit-formation solution.

2. Key Outcomes & Objectives

- **Behavioral Habit Formation** — reduces snooze frequency and builds long-term wake-up consistency.
- **Adaptive Cognitive Warm-up** — calibrates puzzle difficulty (Beginner → Expert) based on historical solve speed and accuracy.
- **Quantified Habit Tracking** — calculates a daily Habit Score stored in PostgreSQL using a weighted formula.
- **Personalized AI Recommendations** — delivers targeted productivity protocols based on a 7-day weekly analysis of user performance.
- **Role-Based Workflows** — tailored interfaces for Users, Wellness Coaches, and Administrators.
- **Executive Reporting** — one-click download of performance reports (Habit Score breakdown, challenge performance, productivity insights, itemized logs).

3. System Architecture

CogniWell follows a layered architecture: a React-based single-page client communicates over a JWT-authenticated REST API with a Node.js / Express server, which orchestrates core services and engines and persists all data in PostgreSQL.



Layer	Responsibility
Client Layer	React 19 + Vite SPA; renders dashboards, alarm UI, and challenge interface.
API Server Layer	Node.js / Express REST API; handles routing, JWT auth, and request validation.
Services & Engines	Auth/RBAC, Alarm Scheduler, Challenge Engine, Weighted Scoring Model, Recommendation Engine, Report Exporter.
PostgreSQL Database	Stores users, alarms, alarm_sessions, "Weighted Scoring Model", coach_messages, user_coach_assignments.

4. Core Modules & Features

4.1 User Authentication & Role-Based Access Control (RBAC)

Secure registration and login using bcrypt password hashing and JWT token authentication, with three distinct system roles:

- **User** — manages alarms, solves challenges, views personal habit scores, productivity insights, and messages their coach.
- **Wellness Coach** — monitors assigned client metrics, analyzes habit trends, and sends personalized guidance.
- **Administrator** — oversees system-wide platform metrics, user management, and service health monitoring.

4.2 Alarm Scheduling System

Supports **Daily**, **Weekday**, **Weekend**, **One-Time**, and **Smart Adaptive** alarm types, with customizable title, trigger time, repeat schedule, challenge theme, difficulty level, sound, and snooze duration limit.

4.3 Cognitive Challenge Engine

Presents Math Exercises, Logic Puzzles, Memory Match, Word Games, Pattern Recognition, and Riddles across five difficulty tiers (Beginner → Expert), with real-time timer tracking, answer verification, a 1–5 star wakefulness rating, and anti-snooze enforcement.

4.4 PostgreSQL Weighted Scoring Model

CogniWell calculates a daily **Habit Score**, stored in the PostgreSQL table *"Weighted Scoring Model"*:

$$\begin{aligned} \text{Habit Score} = & (\text{Wake-Up Consistency} \times 35\%) + (\text{Challenge Completion Success} \times \\ & 25\%) \\ & + (\text{Snooze Reduction} \times 20\%) + (\text{Sleep Schedule Adherence} \times 20\%) \end{aligned}$$

- **Storage Unit** — one row per day per user (UNIQUE constraint on user_id + score_date).
- **Trigger** — automatically recalculated upon every alarm dismissal.

4.5 Recommendation Engine (7-Day Weekly Analysis)

Analyzes each user's past 7 days of Weighted Scoring Model records and alarm sessions to trigger personalized, rule-based recommendations:

Condition	Recommendation Triggered
Low Wake-Up Consistency (<80%)	Circadian Wake Anchor Protocol
High Wake-Up Consistency (≥80%)	Peak Focus Window Allocation (09:00–11:30 AM)
Low Snooze Resistance (<80% / snoozes ≥2)	Snooze Elimination & Barrier Protocol
High Snooze Resistance (≥80%)	Early Sunlight & Neural Hydration Protocol
Low Challenge Solve Rate (<80%)	Adaptive Cognitive Challenge Calibration
High Challenge Solve Rate (≥80%)	Progressive Challenge Tier Optimization

A notification bell displays unread coach messages and active recommendations. Clicking a recommendation navigates to *Active Productivity Recommendations* in Productivity Insights; clicking a message navigates to *Message History* in Wellness Coach Communication.

4.6 Dashboards & Behavioral Analytics

- **User Dashboard** — wake-up statistics, active alarms, recent session logs, streak counter, report export.
- **Habit Score Analytics** (/user/habit-score) — interactive chronological SVG trend graph and 30-day data table.
- **Challenge Performance** (/user/challenge-performance) — solve accuracy, average solve latency, theme breakdown.
- **Productivity Insights** (/user/productivity-insights) — composite productivity index, execution-speed improvement, circadian correlation, and 3 active weekly AI recommendations.

4.7 Wellness Coach Communication

Direct bidirectional messaging between users and assigned Wellness Coaches, with real-time unread badge indicators and direct navigation from notifications.

4.8 Executive Reports & Export System

One-click report download (Weekly / Monthly / Yearly) including:

- Executive summary of metrics (Wake-Up 35%, Challenge 25%, Snooze 20%, Sleep 20%).
- Challenge performance — solve accuracy, average solve time, streak length, snooze counts.
- Productivity insights — composite productivity score, improvement rate, circadian correlation, active weekly recommendations.
- Detailed session logs — itemized table of dates, alarm titles, questions, user answers, solve status, time taken, and wakefulness ratings.

5. Tech Stack

Layer	Technologies Used
Frontend	React 19, Vite 8, React Router DOM 7, CSS3, HTML5
Backend	Node.js, Express 5, REST API
Database	PostgreSQL 16+ (pg connection pool)
Authentication	JSON Web Tokens (JWT), bcryptjs
Data Export	Dynamic CSV / Report Generation Engine
Deployment	Vercel (Frontend), Render / Railway (Backend), Neon / Supabase (Cloud PostgreSQL)

6. Project Structure (Frontend)

```
dashboard/  
src/  
  config/api.js — API base URL & auth header configuration  
  context/AlarmContext.jsx — global alarm triggering & audio handling  
  context/AuthContext.jsx — authentication state & JWT handling  
  pages/AdminDashboard.jsx, CoachDashboard.jsx, UserDashboard.jsx  
  pages/HabitScorePage.jsx, ChallengePerformancePage.jsx, ProductivityInsightsPage.jsx  
  pages/LandingPage.jsx, LoginPage.jsx  
  App.jsx — client route definitions | main.jsx — entry point  
  server/server.js — Express API server, PostgreSQL queries & WSM engine  
  package.json, vercel.json, vite.config.js
```

7. API Documentation

Authentication APIs

Method	Endpoint	Description
POST	/api/auth/register	Register a new user account
POST	/api/auth/login	Login user & receive JWT token
GET	/api/auth/me	Fetch authenticated user profile

Alarm Management APIs

Method	Endpoint	Description
GET	/api/alarms	List user's scheduled alarms
POST	/api/alarms	Create a new cognitive alarm
PUT	/api/alarms/:id	Update an existing alarm
DELETE	/api/alarms/:id	Delete an alarm
PATCH	/api/alarms/:id/enable	Enable alarm
PATCH	/api/alarms/:id/disable	Disable alarm

Alarm Sessions & Verification

Method	Endpoint	Description
POST	/api/alarm-sessions/trigger	Trigger active alarm session
POST	/api/alarm-sessions/complete	Validate challenge answer & log dismissal

Weighted Scoring Model & Recommendations

Method	Endpoint	Description
GET	/api/user/weighted-scoring-model	Fetch 30-day Habit Score history from PostgreSQL
POST	/api/user/weighted-scoring-model/sync	Recalculate today's score & upsert row
GET	/api/user/recommendations	Fetch dynamic recommendations from 7-day weekly analysis

8. Installation & Local Setup

- **Prerequisites** — Node.js v18+, npm v9+, PostgreSQL v14+.
- Clone the repository and run `npm install` inside the `dashboard/` directory.
- Configure a `.env` file with `PORT`, `JWT_SECRET`, and PostgreSQL credentials (`PGUSER`, `PGPASSWORD`, `PGHOST`, `PGPORT`, `PGDATABASE`) or a single `DATABASE_URL`.
- Create the database (`CREATE DATABASE cogniwell_db;`) — the backend auto-creates all required tables on server start.
- Run `npm run server` (backend on localhost:5001) and `npm run dev` (frontend on localhost:5173) in separate terminals.

9. Deployment Guide

Frontend (Vercel)

- Import the repository on Vercel and set Root Directory to `dashboard`.
- Set environment variable `VITE_API_URL` to the backend server URL.
- Deploy — the included `vercel.json` ensures zero SPA routing errors.

Backend & Database

- Host PostgreSQL on Neon.tech or Supabase.
- Host `server/server.js` on Render.com or Railway.app with `DATABASE_URL` and `JWT_SECRET` configured.

10. Conclusion

CogniWell reframes the daily alarm from a passive interruption into an active, measurable behavioral health intervention. By combining a cognitive challenge engine, a transparent weighted scoring model, and a rule-based 7-day recommendation system, the platform gives users, wellness coaches, and administrators a shared, data-driven view of wake-up habits and their downstream effect on productivity. The modular Node.js / Express / PostgreSQL architecture keeps the system straightforward to extend — future iterations can introduce ML-based scoring, richer coach analytics, and deeper wearable integrations without disrupting the current data model.